

INFORME: USO DE GSON PARA GUARDAR ARCHIVOS JSON EN "MazeRunner"

Resumen

- La librería Gson se utiliza para serializar (guardar) y deserializar (cargar) datos en formato JSON.
- Casos principales de uso:
 - Guardado de partidas por usuario en la carpeta **saves**.
 - Guardado de estadísticas globales en un archivo dedicado.
- La escritura se realiza en UTF-8 y se crean directorios automáticamente cuando es necesario.
- Se evitan referencias circulares en la serialización gracias a campos `transient` (por ejemplo, en `Jugador`).

Dependencia Maven (definida en `pom.xml`)

- Grupo/Artefacto/Versión: `com.google.code.gson : gson : 2.13.2`
- La dependencia permite usar: `Gson`, `GsonBuilder`, `JsonParser`, `JsonObject`, etc.

Guardado de partidas (`ControladorBD.guardar`)

- Clase: `ControladorBD`
- Método: `public static void guardar(Laberinto laberinto)`

Flujo de trabajo

1. Construcción de un objeto `Gson` por defecto para serializar.
2. Resolución de la ruta de salida:
 - Raíz del proyecto: `System.getProperty("user.dir")`.
 - Subcarpeta: **saves** (se crea si no existe).
 - Nombre de archivo por usuario:
`laberinto-<email-sanitizado>.json`.
 - Se usa un método de sanitización para reemplazar caracteres inválidos en Windows por guión bajo.
3. Escritura del JSON:
 - Se abre un `FileOutputStream` envuelto por `OutputStreamWriter` en UTF-8.
 - Se usa `JsonWriter` para que `Gson` escriba el objeto:
`gson.toJson(laberinto, Laberinto.class, writer)`.

- Se realiza `flush` y se informa la ruta completa por consola.
- 4. Manejo de errores: se captura `IOException` y se registra el problema.

Consideración clave: estructuras cíclicas

- Se marca como `transient` el campo `Jugador.celdaActual` para que Gson lo omita y así evitar ciclos de referencia (`Celda → Entidad → Jugador → Celda...`).

Guardado de estadísticas (`Statistics.save`)

- Clase: `Statistics`
- Método: `private static synchronized void save(Statistics s)`

Flujo de trabajo

1. Archivo destino: `saves/stats.json` (misma carpeta persistente donde se guardan las partidas).
2. Se asegura la existencia del directorio `saves` (`makedirs` si no existe).
3. Se utiliza Gson con *pretty printing* para obtener un JSON legible por humanos: `new GsonBuilder().setPrettyPrinting().create()`
4. Se serializa únicamente el mapa `players` (`email → PlayerStats`), manteniendo el archivo simple y directo.
5. Escritura en UTF-8 con `OutputStreamWriter` y `flush`.
6. Manejo de errores con salida a `stderr` en caso de falla.

Lectura (deserialización)

`Statistics.load`

- Lee `saves/stats.json` en UTF-8.
- Usa `TypeToken<Map<String, PlayerStats>>` para deserializar el mapa de estadísticas.
- Recalcula el total de partidas (`wins + losses`) tras cargar.

`Laberinto.cargarJson` (sobrecargas)

- Lee el archivo JSON de una partida.
- Usa `JsonParser/JsonObject` para recorrer la estructura y reconstruir:
 - Celdas (valores y contorno del laberinto).
 - Jugador (desde su `JsonObject`) mediante `Jugador.fromJson`.

- o Entidades (Trampa, Enemigo, Llave, Puerta, etc.) con una factoría: `crearEntidadDesdeJson`.
- Recoloca las entidades/celdas en memoria y enlaza referencias (por ejemplo, celda del jugador basada en `posX/posY`).

seedFromSaves (Statistics)

- Recorre archivos de guardado `laberinto-*.json` para enriquecer el resumen de estadísticas mostrado (ejemplo: puntaje máximo), sin persistir cambios.

Rutas y nombres de archivo

- **Partidas por usuario:**
 - o Base: `<raíz del proyecto>/saves/`
 - o Formato: `laberinto-<email-sanitizado>.json`
 - Ejemplo: `laberinto-paris_ucab.com.json`
- **Estadísticas:**
 - o Base: `<raíz del proyecto>/saves/`
 - o Nombre: `stats.json`

Codificación de caracteres

- Lectura y escritura en UTF-8 explícita mediante `StandardCharsets.UTF_8` para evitar problemas con acentos y caracteres especiales.

Prevención de problemas comunes con JSON

- Referencias circulares: uso de campos `transient` para excluir enlaces que crearían ciclos durante la serialización.
- Nombres de archivo seguros: sanitización del correo para generar nombres válidos en Windows.
- Robustez de directorios: creación automática de la carpeta `saves` antes de escribir.

Puntos clave de diseño

- Serialización simple del estado del juego (Laberinto) con Gson; reconstrucción cuidadosa al cargar mediante factorías y validaciones.
- Archivo de estadísticas separado, con formato legible (*pretty printing*) para facilitar inspección y depuración.
- Separación de responsabilidades:

- | | |
|--|---|
| C AESCifrado | C ControladorBD |
| key: Key | |
| <ul style="list-style-type: none"> o AESCifrado(): o Descifrado(String): String o Cifrado(String): String | <ul style="list-style-type: none"> o ControladorBD(): o sanitizeForFile(String): String o guardar(Laberinto): void |

